Bar Chart / Histogram Without a Library

- **Problem:** Build a bar chart component from scratch (no Chart.js, D3, Recharts). It must draw bars from `[{ id, label, value }]`, show a y-axis with round tick values and gridlines, show x-axis labels, resize with its container, show a tooltip on hover and keyboard focus, support negative values and sorting. Then reuse it as a **histogram**: take raw numbers, group them into N equal-width bins and chart the counts.
- **Tech stack:** React (Vite, JavaScript), SVG, plain CSS (default stack, since none was given)
- **Time limit:** 60 minutes (default, since none was given)
- **Often asked at:** Uber, Rippling, Stripe
- **Why it matters:** SVG coordinate system, scales (data space to pixel space), nice axis ticks, responsive measurement, accessible data visualisation

---

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This becomes the build and test checklist.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** Find the single source of truth. Derive everything else.
5. **Design data structures and contracts.** Shape of the data and the component props.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Empty, all zeros, negatives, one value, many bars, tiny width.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state, contracts	7 min
Pure scale math (`linearScale` , `bandScale` , `niceTicks`)	10 min
SVG rendering: bars, axes, labels	13 min
Responsive width, tooltip, keyboard	10 min
Histogram binning + demo controls	10 min

Phase	Time
Edge cases + wrap-up	5 min

Think out loud

Say what you are doing and why, all the time: "I'll keep the math in pure functions so I can test it without a browser", "SVG y grows downward, so I flip the range". The interviewer grades how you reason, not only what runs. If you get stuck, say what you are trying and what you would check next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a vertical bar chart from `data = [{ id, label, value }]`.
- Y-axis with 4-6 **round** tick values (0, 50, 100, not 0, 47.4, 94.8) and horizontal gridlines.
- X-axis label under each bar; skip labels when there is no room so they never overlap.
- Bars scale to fit the container width and redraw on resize.
- Tooltip with label and value on hover **and** on keyboard focus.
- Negative values grow downward from a visible zero line, in a different colour.
- Sort: original / ascending / descending; randomize data to show updates.
- Histogram: paste raw numbers, choose bin count (1-30), chart the counts per bin.
- Empty state when there is no data.

Non-functional requirements

- **Performance:** a few hundred bars render instantly; math is O(n). No re-measuring the DOM per bar.
- **Scalability:** the chart is data-driven; any dataset of the same shape works. Histogram bins thousands of values in one pass.
- **Error handling:** invalid input numbers are skipped and counted, never crash; all-zero or single-value data must not divide by zero.
- **Accessibility:** each bar is focusable with an `aria-label` like `"Jul: 237"`; the chart has a name; focus ring visible; respects `prefers-reduced-motion`.
- **Reusability:** one `BarChart` component serves both the bar chart and the histogram through props.
- **Extensibility:** scales are pure functions, so horizontal bars, grouped bars or a line chart reuse the same math.

Clarifying questions to ask

1. Vertical bars only, or horizontal too? Grouped / stacked series?
2. Can values be negative? Can they be decimals?
3. Fixed size or should it fill its container and respond to resize?
4. How many bars at most? (12 months vs 10,000 data points changes SVG vs Canvas.)
5. Do we need animation on data change, and a tooltip on hover?
6. For the histogram, does the user choose the bin count or should we compute it (Sturges' rule)?

Assumptions we will build with

- Vertical bars, one series. Values may be negative or decimal.
- Fills container width, fixed height (320px).
- Up to a few hundred bars, so SVG (DOM nodes) is fine.
- User picks histogram bin count; bins are equal-width; the max value belongs to the last bin.
- No chart library and no D3; only React, SVG and CSS.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Revenue chart: 12 bars, ticks `0 50 100 150 200 250`, month labels
Hover a bar	Bar highlights, tooltip above it shows `Jul / 237`
Hover a zero-height bar	Still shows tooltip (full-height hit area)
Tab to a bar	Focus ring, tooltip shown, screen reader reads `"Jul: 237"`
Mouse leave / blur	Tooltip hides
Sort = Descending	Bars reorder from largest to smallest, labels follow, ticks unchanged
Switch dataset to Profit / loss	Negative bars in red go below the zero line; ticks include negatives
Click Randomize	New values, axis recomputes nice ticks, bars animate to new heights
Resize window / phone width	Chart redraws at the new width; labels skip to avoid overlap
Switch to Histogram	150 sample values in 10 bins, touching bars, range labels like `100–125.4`
Drag bins slider to 30	30 bins; only every Nth label shown
Type `5 5 5 abc`	One bin with count 3, warning `1 invalid ignored`
Clear the textarea	"No data to display" empty state

3. Design the architecture

Module	Responsibility
`chart/scales.js`	Pure math: `linearScale`, `bandScale`, `niceStep`, `niceTicks`
`chart/histogram.js`	Pure: `parseNumbers(text)`, `binValues(values, binCount)`
`chart/useElementWidth.js`	Hook: measure container width with `ResizeObserver`
`chart/BarChart.jsx`	Reusable component: SVG axes, bars, labels, hit areas, tooltip
`demos/BarDemo.jsx`	Dataset picker, sort, randomize; owns bar data
`demos/HistogramDemo.jsx`	Textarea + bin slider; turns raw numbers into bar data
`App.jsx`	Tab switch between the two demos

```
App
├── BarDemo ──────────────┐
│     data, sort          │  data = [{ id, label, value }]
└── HistogramDemo ────────┤
      text -> parseNumbers -> binValues
                          ▼
                    BarChart
                    ├── useElementWidth  (container width)
                    ├── niceTicks / linearScale / bandScale  (math)
                    └── <svg> ticks + bars + labels + hit rects, <div> tooltip
```

Start with everything in `BarChart`; the scales are pulled out first because they are the part you want to unit-test. The histogram reuses `BarChart` by mapping bins to `{ id, label, value }`, so there is no second chart component.

4. Design the state / data model

State or field	Owner	Purpose
`width`	`useElementWidth`	Container width in px, drives the x scale
`activeId`	`BarChart`	Which bar is hovered / focused (for tooltip + highlight)
`key`	`BarDemo`	Selected dataset (`revenue` / `profit`)
`series`	`BarDemo`	Values of each dataset (Randomize updates it)
`sort`	`BarDemo`	`none` / `asc` / `desc`
`text`	`HistogramDemo`	Raw textarea input
`binCount`	`HistogramDemo`	Number of bins (1-30)

```
// BarChart
const [ref, width] = useElementWidth();
const [activeId, setActiveId] = useState(null);

// BarDemo
const [key, setKey] = useState('revenue');
const [series, setSeries] = useState({ revenue: REVENUE, profit: PROFIT });
const [sort, setSort] = useState('none');

// HistogramDemo
const [text, setText] = useState(SAMPLE);
const [binCount, setBinCount] = useState(10);
```

Senior note: derive, don't store.

- Ticks, domain, scales, bar positions, label spacing: all derived from `data` + `width` on every render. Storing them would mean keeping them in sync.
- Sorted data is derived (`useMemo`), so "Original" order is never lost.
- Parsed values and bins are derived from `text` + `binCount`; the only stored thing is what the user typed.
- `activeId` stores the **id**, not the index, so the tooltip still points at the right bar after a sort.
- The chart owns only UI state (hover, width). The data owner is the parent; `BarChart` is a controlled, data-in component.

5. Design the data structures and contracts

Chart input

```
[
  { id: 'Jan', label: 'Jan', value: 120 },
  { id: 'Feb', label: 'Feb', value: 95 },
  // ...
]
```

`id` is a stable key. React uses it to keep the same `<rect>` when bars reorder, which lets the CSS transition slide the bar instead of re-creating it. `label` is display text; `value` is any finite number.

Histogram bin

```
{ x0: 100, x1: 125.4, count: 5 }  // binValues output
// mapped to chart input:
{ id: 0, label: '100–125.4', value: 5 }
```

`BarChart` props contract

Prop	Type	Default	Meaning
`data`	`{ id, label, value }[]`	required	Bars, in display order
`title`	string	required	Caption and accessible name
`height`	number	320	Chart height in px
`padding`	0-1	0.2	Gap share of each slot (0.04 for histograms)
`minLabelWidth`	number	48	Min px per label before labels are skipped
`format`	`(n) => string`	`Intl.NumberFormat`	Tick, tooltip and aria value format

Pure function contracts

```
linearScale([0, 250], [300, 0])(125)   // -> 150   (data -> pixel)
bandScale(4, 400, 0.2)                 // x(i) -> 10, 110...; x.step 100; x.bandwidth 80
niceTicks([120, 237, 80])              // -> { min: 0, max: 250, ticks: [0, 50, 100, 150, 200, 250] }
```

```
parseNumbers('1, 2 abc,3')            // -> { values: [1, 2, 3], invalid: 1 }
binValues([0..10], 5)                 // -> counts [2, 2, 2, 2, 3]
```

No API here; data is local. If it came from a server, the contract stays the same and the parent maps the response into `{ id, label, value }`.

6. Trace the data flow

User clicks Randomize on the revenue chart:

```
click "Randomize"
  │
  ▼
setSeries(s => ({ ...s, revenue: s.revenue.map(d => ({ ...d, value: rand() })) }))
  │   ids unchanged, values new
  ▼
BarDemo re-renders → useMemo sorts (if sort != none) → data
  │
  ▼
BarChart(data)
  │  niceTicks(values)          → new { min, max, ticks }, e.g. 0..250 → 0..300
  │  linearScale([min,max],[innerH,0]) → y()
  │  bandScale(n, innerW, 0.2)  → x(), unchanged (same count)
  ▼
React diff by key=id → same <rect> nodes, new y/height attributes
  │
  ▼
CSS `transition: y .3s, height .3s` animates bars; tick <g> keyed by value re-render
```

7. Important technical challenges

Challenge 1: Mapping data to pixels (scales) with a flipped y-axis

Problem. SVG `y = 0` is the **top**. A value of 237 in a 268px-tall plot must end up near the top, and its rect must start at that y and extend down to the zero line. With negative values (profit -25) the bar must start at zero and go **down**.

Solution. A linear scale is just a line through two points: `(d0, r0)` and `(d1, r1)`. Flip the range (`[innerH, 0]`) so bigger values go up. For each bar, the top is `y(max(0, v))` and the height is `|y(v) - y(0)|`, which handles both signs.

```
export function linearScale([d0, d1], [r0, r1]) {
  const k = (r1 - r0) / (d1 - d0);
  return (v) => r0 + (v - d0) * k;
}
```

```
const y = linearScale([min, max], [innerH, 0]);
<rect y={y(Math.max(0, d.value))} height={Math.abs(y(d.value) - y(0))} />
```

The x-axis is a **band** scale: the width is split into equal slots, and a bar takes `1 - padding` of its slot, centered.

```
export function bandScale(count, width, padding = 0.2) {
  const step = count ? width / count : 0;
  const bandwidth = step * (1 - padding);
  const x = (i) => i * step + (step - bandwidth) / 2;
  x.step = step;
  x.bandwidth = bandwidth;
  return x;
}
```

Challenge 2: "Nice" axis ticks

Problem. Dividing 0-237 into 5 gives ticks `0, 47.4, 94.8...`, which nobody can read. Also: all values 0 gives domain `[0, 0]`, and `(v - d0) / 0` is `NaN`. Floats give `0.30000000000000004`.

Solution. Round the raw step up to 1, 2, 5 or 10 times a power of ten, then snap min down and max up to multiples of that step. Always include 0 so bar lengths are honest. Guard the zero-width domain. Clean float noise with `toPrecision`.

```
export function niceStep(span, count) {
  const raw = span / count;                        // 237 / 5 = 47.4
  const pow = 10 ** Math.floor(Math.log10(raw));   // 10
  const f = raw / pow;                             // 4.74
  return (f <= 1 ? 1 : f <= 2 ? 2 : f <= 5 ? 5 : 10) * pow; // 50
}

export function niceTicks(values, count = 5) {
  let min = Math.min(0, ...values);
  let max = Math.max(0, ...values);
  if (min === max) max = 1;
  const step = niceStep(max - min, count);
  min = Math.floor(min / step) * step;
  max = Math.ceil(max / step) * step;
  const ticks = [];
  for (let t = min; t <= max + step / 2; t += step) ticks.push(+t.toPrecision(12));
  return { min, max, ticks };
}
```

Challenge 3: Responsive width without distorting text

Problem. `viewBox` + `width="100%"` scales the whole drawing, so on a phone the 12px labels shrink to 5px, and on a wide screen bars get fat. We want bars to re-layout, not zoom.

Solution. Measure the container with `ResizeObserver` and draw in real pixels. Render the SVG only once `width > 0`. Then skip x labels when slots are too narrow: show every `ceil(minLabelWidth / step)`-th label.

```
export function useElementWidth() {
  const ref = useRef(null);
  const [width, setWidth] = useState(0);
  useLayoutEffect(() => {
    const observer = new ResizeObserver(([entry]) => setWidth(entry.contentRect.width));
    observer.observe(ref.current);
    return () => observer.disconnect();
  }, []);
  return [ref, width];
}

const labelEvery = Math.max(1, Math.ceil(minLabelWidth / x.step));
{i % labelEvery === 0 && <text>{d.label}</text>}
```

Challenge 4: Tooltip and accessibility for tiny bars

Problem. A value of 0 has a 0px-tall bar: you cannot hover it. A screen reader sees an SVG full of anonymous rects. The tooltip must be HTML (text wrapping, no SVG clipping) but placed over SVG coordinates.

Solution. Draw a transparent **full-height hit rect** per slot on top of each bar. It gets the mouse/focus handlers, `tabIndex={0}`, `role="img"` and an `aria-label`. The tooltip is an absolutely positioned `<div>` inside the same `position: relative` box, so the same scale values give its position. It is `aria-hidden` because the focused rect already announces the value.

```
<rect
  className="hit"
  x={i * x.step} width={x.step} height={innerH}
  tabIndex={0} role="img" aria-label={`${d.label}: ${format(d.value)}`}
  onMouseEnter={() => setActiveId(d.id)} onMouseLeave={() => setActiveId(null)}
  onFocus={() => setActiveId(d.id)} onBlur={() => setActiveId(null)}
/>

<div className="tooltip" aria-hidden="true"
  style={{ left: M.left + x(i) + x.bandwidth / 2, top: M.top + y(Math.max(0, v)) }} />
/* .tooltip { position: absolute; transform: translate(-50%, calc(-100% - 8px)); } */
```

Challenge 5 (histogram): Binning edge cases

Problem. With 10 bins over `[0, 10]`, `floor((10 - 0) / 1) = 10`, which is an 11th bin that does not exist. If every value is the same, the bin width is 0 and you divide by zero.

Solution. Clamp the index to the last bin, and return a single bin when `min === max`. One pass, O(n), no sorting needed. Use a loop for min/max (spreading 1M values into `Math.min` overflows the stack).

```
if (min === max) return [{ x0: min, x1: max, count: values.length }];
const width = (max - min) / binCount;
for (const v of values) bins[Math.min(binCount - 1, Math.floor((v - min) / width))].count++;
```

8. Implement in the correct order

#	Step	Working result
1	Hard-coded 12 `<rect>`s in an SVG with fixed width 600	Bars on screen
2	`linearScale` + `bandScale`, bars from data	Bars sized by value
3	`niceTicks` + gridlines + y labels	Readable y-axis
4	X labels under bars	Full static chart
5	Unit-test the scales with `node:assert`	Math trusted
6	`useElementWidth` + margins	Responsive chart
7	Label skipping by slot width	No overlap on phones
8	Hit rects + `activeId` + tooltip	Hover works
9	Focus handlers, `aria-label`, focus ring	Keyboard + screen reader
10	Negative values: top/height formula, red class, zero line	Profit chart
11	Demo controls: dataset, sort (derived), randomize, CSS transitions	Interactive bar demo
12	`parseNumbers` + `binValues` + histogram demo	Histogram reuses `BarChart`

9. Cover all states and edge cases

State / edge case	What happens
Initial (width not yet measured)	Container is rendered, SVG waits for `width > 0` (avoids a 0px flash with NaN)
Normal data	Bars, nice ticks, labels
Empty data `[]`	"No data to display" placeholder, same height so layout doesn't jump
All zeros / `niceTicks([])`	Domain becomes `[0, 1]`, flat bars, no `NaN`
Single bar	One slot = full width, padding keeps it from touching edges
Negative values	Bar from zero line downward, red, ticks include negatives, darker zero line
Only negative values	Domain `[min, 0]`, bars hang from the top zero line
Zero value	0px bar but full-height hit area still shows tooltip
Many bars / narrow screen	Every Nth label shown; bars thin but present
Decimal values	`Intl.NumberFormat` max 2 decimals; ticks use `toPrecision` to strip float noise
Huge values (1e9)	Nice step works at any magnitude (power of ten)

State / edge case	What happens
Sorting	Derived copy; original order restorable; keyed rects slide
Data changes while hovering	`activeId` lookup by id; removed id simply hides the tooltip
Tooltip near edge	`left` clamped to `[50, width - 50]`
Histogram: invalid tokens	Skipped and counted: "1 invalid ignored"
Histogram: all equal values	One bin with full count
Histogram: max value	Clamped into the last bin
Reduced motion	Transitions disabled via media query

10. Final structure

```
src/
├── main.jsx
├── App.jsx                # tab switch
├── index.css              # chart + demo styles
├── chart/
│   ├── scales.js          # linearScale, bandScale, niceStep, niceTicks (pure)
│   ├── histogram.js       # parseNumbers, binValues (pure)
│   ├── useElementWidth.js # ResizeObserver hook
│   ├── BarChart.jsx       # reusable SVG bar chart
│   └── chart.test.mjs     # node assert self-check
└── demos/
    ├── BarDemo.jsx        # dataset, sort, randomize
    └── HistogramDemo.jsx  # textarea + bins -> BarChart
```

11. Full working code

Setup: `npm create vite@latest bar-chart -- --template react-swc`, replace `src/` with the files below, `npm run dev`. Self-check: `node src/chart/chart.test.mjs`.

`src/chart/scales.js`

```
// Pure math: data values -> pixel positions. No React, no DOM, easy to test.

export function linearScale([d0, d1], [r0, r1]) {
  const k = (r1 - r0) / (d1 - d0);
  return (v) => r0 + (v - d0) * k;
}

// Splits `width` into `count` equal slots; each bar takes (1 - padding) of its slot.
export function bandScale(count, width, padding = 0.2) {
  const step = count ? width / count : 0;
```

```
  const bandwidth = step * (1 - padding);
  const x = (i) => i * step + (step - bandwidth) / 2;
  x.step = step;
  x.bandwidth = bandwidth;
  return x;
}

// Rounds a raw step up to 1, 2, 5 or 10 times a power of ten (e.g. 47.4 -> 50).
export function niceStep(span, count) {
  const raw = span / count;
  const pow = 10 ** Math.floor(Math.log10(raw));
  const f = raw / pow;
  return (f <= 1 ? 1 : f <= 2 ? 2 : f <= 5 ? 5 : 10) * pow;
}

// Domain always includes 0 so bar heights stay honest, then snaps to nice round ticks.
export function niceTicks(values, count = 5) {
  let min = Math.min(0, ...values);
  let max = Math.max(0, ...values);
  if (min === max) max = 1; // no data or all zeros: avoid a 0-wide domain (divide by zero)
  const step = niceStep(max - min, count);
  min = Math.floor(min / step) * step;
  max = Math.ceil(max / step) * step;
  const ticks = [];
  // toPrecision strips float noise like 0.30000000000000004
  for (let t = min; t <= max + step / 2; t += step) ticks.push(+t.toPrecision(12));
  return { min, max, ticks };
}
```

src/chart/histogram.js

```
// "12, 15 abc\n20" -> { values: [12, 15, 20], invalid: 1 }
export function parseNumbers(text) {
  const tokens = text.split(/[\s,]+/).filter(Boolean);
  const values = tokens.map(Number).filter(Number.isFinite);
  return { values, invalid: tokens.length - values.length };
}

// Equal-width bins over [min, max]. The max value goes in the last bin, not a new one.
export function binValues(values, binCount) {
  if (values.length === 0) return [];
  let min = Infinity;
  let max = -Infinity;
  for (const v of values) {
    if (v < min) min = v;
    if (v > max) max = v;
  }
```

```
  if (min === max) return [{ x0: min, x1: max, count: values.length }];

  const width = (max - min) / binCount;
  const bins = Array.from({ length: binCount }, (_, i) => ({
    x0: min + i * width,
    x1: i === binCount - 1 ? max : min + (i + 1) * width,
    count: 0,
  }));
  for (const v of values) bins[Math.min(binCount - 1, Math.floor((v - min) / width))].count++;
  return bins;
}
```

src/chart/useElementWidth.js

```
import { useLayoutEffect, useRef, useState } from 'react';

// Measures a container so the SVG can be drawn in real pixels (viewBox scaling would stretch text).
export function useElementWidth() {
  const ref = useRef(null);
  const [width, setWidth] = useState(0);

  useLayoutEffect(() => {
    const el = ref.current;
    if (!el) return;
    const observer = new ResizeObserver(([entry]) => setWidth(entry.contentRect.width));
    observer.observe(el); // fires once right away with the initial size
    return () => observer.disconnect();
  }, []);

  return [ref, width];
}
```

src/chart/BarChart.jsx

```
import { useState } from 'react';
import { bandScale, linearScale, niceTicks } from './scales';
import { useElementWidth } from './useElementWidth';

const M = { top: 16, right: 16, bottom: 36, left: 52 };
const defaultFormat = new Intl.NumberFormat('en', { maximumFractionDigits: 2 }).format;

export default function BarChart({
  data, // [{ id, label, value }]
  title,
  height = 320,
  padding = 0.2,
```

```
  minLabelWidth = 48, // narrower slots skip labels so they never overlap
  format = defaultFormat,
}) {
  const [ref, width] = useElementWidth();
  const [activeId, setActiveId] = useState(null);

  const innerW = Math.max(0, width - M.left - M.right);
  const innerH = height - M.top - M.bottom;
  const { min, max, ticks } = niceTicks(data.map((d) => d.value));
  const y = linearScale([min, max], [innerH, 0]); // SVG y grows downward, so the range is flipped
  const x = bandScale(data.length, innerW, padding);
  const labelEvery = Math.max(1, Math.ceil(minLabelWidth / x.step));

  // Store the id, not the index: after a sort or data change the tooltip follows the right bar.
  const activeIndex = data.findIndex((d) => d.id === activeId);
  const active = data[activeIndex];

  return (
    <figure className="chart">
      <figcaption>{title}</figcaption>
      <div ref={ref} className="chart-box" style={{ height }}>
        {data.length === 0 ? (
          <p className="chart-empty">No data to display</p>
        ) : (
          width > 0 && (
            <svg width={width} height={height} role="group" aria-label={title}>
              <g transform={`translate(${M.left},${M.top})`}>
                {ticks.map((t) => (
                  <g key={t} className={t === 0 ? 'tick zero' : 'tick'} transform=
{`translate(0,${y(t)})`}>
                    <line x2={innerW} />
                    <text x={-8} dy="0.32em" textAnchor="end">
                      {format(t)}
                    </text>
                  </g>
                ))}
                {data.map((d, i) => (
                  <g key={d.id} className={d.id === activeId ? 'col active' : 'col'}>
                    <rect
                      className={d.value < 0 ? 'bar neg' : 'bar'}
                      x={x(i)}
                      y={y(Math.max(0, d.value))}
                      width={x.bandwidth}
                      height={Math.abs(y(d.value) - y(0))}
                    />
                    {i % labelEvery === 0 && (
                      <text className="x-label" x={x(i) + x.bandwidth / 2} y={innerH + 20}
```

```
textAnchor="middle">
                        {d.label}
                      </text>
                    )}
                    {/* full-height hit area: zero or tiny bars are still easy to hover and focus */}
                    <rect
                      className="hit"
                      x={i * x.step}
                      width={x.step}
                      height={innerH}
                      tabIndex={0}
                      role="img"
                      aria-label={`${d.label}: ${format(d.value)}`}
                      onMouseEnter={() => setActiveId(d.id)}
                      onMouseLeave={() => setActiveId(null)}
                      onFocus={() => setActiveId(d.id)}
                      onBlur={() => setActiveId(null)}
                    />
                  </g>
                ))}
              </g>
            </svg>
          )
        )}
        {active && (
          <div
            className="tooltip"
            aria-hidden="true" // the focused bar's aria-label already announces this
            style={{
              left: Math.min(Math.max(M.left + x(activeIndex) + x.bandwidth / 2, 50), width - 50),
              top: M.top + y(Math.max(0, active.value)),
            }}
          >
            <strong>{active.label}</strong>
            {format(active.value)}
          </div>
        )}
      </div>
    </figure>
  );
}
```

src/demos/BarDemo.jsx

```
import { useMemo, useState } from 'react';
import BarChart from '../chart/BarChart';
```

```
const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
const QUARTERS = ["Q1 '24", "Q2 '24", "Q3 '24", "Q4 '24", "Q1 '25", "Q2 '25", "Q3 '25", "Q4 '25"];
const toSeries = (labels, values) => labels.map((label, i) => ({ id: label, label, value: values[i]
}));

const DATASETS = {
  revenue: {
    title: 'Monthly revenue (k$)',
    range: [20, 240],
    data: toSeries(MONTHS, [120, 95, 140, 180, 160, 210, 237, 190, 150, 130, 175, 220]),
  },
  profit: {
    title: 'Quarterly profit / loss (k$)',
    range: [-80, 120],
    data: toSeries(QUARTERS, [40, -25, 60, 15, -10, 85, 0, 52]),
  },
};

const SORTERS = {
  none: null,
  asc: (a, b) => a.value - b.value,
  desc: (a, b) => b.value - a.value,
};

const randInt = ([lo, hi]) => Math.round(lo + Math.random() * (hi - lo));

export default function BarDemo() {
  const [key, setKey] = useState('revenue');
  const [series, setSeries] = useState({ revenue: DATASETS.revenue.data, profit: DATASETS.profit.data
});
  const [sort, setSort] = useState('none');

  // Sorting is derived at render time; the stored data keeps its original order.
  const data = useMemo(() => {
    const compare = SORTERS[sort];
    return compare ? [...series[key]].sort(compare) : series[key];
  }, [series, key, sort]);

  const randomize = () =>
    setSeries((s) => ({ ...s, [key]: s[key].map((d) => ({ ...d, value: randInt(DATASETS[key].range) }))
}));

  return (
    <section>
      <div className="controls">
        <label>
          Dataset{' '}
```

```
          <select value={key} onChange={(e) => setKey(e.target.value)}>
            <option value="revenue">Revenue</option>
            <option value="profit">Profit / loss</option>
          </select>
        </label>
        <label>
          Sort{' '}
          <select value={sort} onChange={(e) => setSort(e.target.value)}>
            <option value="none">Original</option>
            <option value="asc">Ascending</option>
            <option value="desc">Descending</option>
          </select>
        </label>
        <button type="button" onClick={randomize}>
          Randomize
        </button>
      </div>
      <BarChart data={data} title={DATASETS[key].title} />
    </section>
  );
}
```

src/demos/HistogramDemo.jsx

```
import { useMemo, useState } from 'react';
import BarChart from '../chart/BarChart';
import { binValues, parseNumbers } from '../chart/histogram';

// Deterministic sample: 150 "response times" in ms, roughly bell-shaped (sum of 3 uniforms).
let seed = 7;
const rand = () => (seed = (seed * 16807) % 2147483647) / 2147483647;
const SAMPLE = Array.from({ length: 150 }, () => Math.round(80 + (rand() + rand() + rand()) *
100)).join(', ');

const fmt = new Intl.NumberFormat('en', { maximumFractionDigits: 1 }).format;

export default function HistogramDemo() {
  const [text, setText] = useState(SAMPLE);
  const [binCount, setBinCount] = useState(10);

  const { values, invalid } = useMemo(() => parseNumbers(text), [text]);
  const data = useMemo(
    () => binValues(values, binCount).map((b, i) => ({ id: i, label: `${fmt(b.x0)}–${fmt(b.x1)}`,
value: b.count })),
    [values, binCount],
  );
```

```
  return (
    <section>
      <label className="field">
        Values (comma or space separated)
        <textarea rows={4} value={text} onChange={(e) => setText(e.target.value)} />
      </label>
      <div className="controls">
        <label>
          Bins: {binCount}{' '}
          <input type="range" min={1} max={30} value={binCount} onChange={(e) =>
setBinCount(+e.target.value)} />
        </label>
        <span className="muted">{values.length} values</span>
        {invalid > 0 && <span className="warn">{invalid} invalid ignored</span>}
      </div>
      <BarChart data={data} title="Response time distribution (ms)" padding={0.04} minLabelWidth={80}
format={fmt} />
    </section>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import BarDemo from './demos/BarDemo';
import HistogramDemo from './demos/HistogramDemo';

const TABS = { bar: 'Bar chart', histogram: 'Histogram' };

export default function App() {
  const [tab, setTab] = useState('bar');

  return (
    <main className="app">
      <h1>Charts without a library</h1>
      <div className="controls" role="group" aria-label="Chart type">
        {Object.entries(TABS).map(([key, label]) => (
          <button key={key} type="button" aria-pressed={tab === key} onClick={() => setTab(key)}>
            {label}
          </button>
        ))}
      </div>
      {tab === 'bar' ? <BarDemo /> : <HistogramDemo />}
    </main>
  );
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`src/index.css`

```
:root {
  --bar: #4f6bed;
  --neg: #e5484d;
  --grid: #e3e6ea;
  --axis: #8c959f;
  --text: #1f2328;
  --muted: #656d76;
  font-family: system-ui, sans-serif;
  color: var(--text);
  background: #fff;
}

body { margin: 0; }
.app { max-width: 880px; margin: 0 auto; padding: 24px 16px; }
.controls { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; margin-bottom: 16px; }
button, select { font: inherit; padding: 6px 12px; border: 1px solid #c9ced6; border-radius: 6px;
background: #fff; cursor: pointer; }
button[aria-pressed='true'] { background: var(--bar); border-color: var(--bar); color: #fff; }
.field { display: grid; gap: 4px; margin-bottom: 12px; }
textarea { font: 13px/1.4 ui-monospace, monospace; }
.muted { color: var(--muted); }
.warn { color: #9a6700; }

.chart { margin: 0; }
.chart figcaption { font-weight: 600; margin-bottom: 8px; }
.chart-box { position: relative; width: 100%; }
.chart-box svg { display: block; }
.chart-empty { display: grid; place-items: center; height: 100%; margin: 0; color: var(--muted);
border: 1px dashed #c9ced6; border-radius: 8px; }

.tick line { stroke: var(--grid); }
```

```
.tick.zero line { stroke: var(--axis); }
.tick text, .x-label { font-size: 12px; fill: var(--muted); }
.bar { fill: var(--bar); transition: x 0.3s, y 0.3s, width 0.3s, height 0.3s; }
.bar.neg { fill: var(--neg); }
.x-label { transition: x 0.3s; }
.col.active .bar { opacity: 0.8; }
.hit { fill: transparent; outline: none; }
.col.active .hit { fill: rgb(0 0 0 / 0.04); }
.hit:focus-visible { stroke: var(--text); stroke-width: 2; }

.tooltip {
  position: absolute;
  transform: translate(-50%, calc(-100% - 8px));
  display: grid;
  pointer-events: none;
  white-space: nowrap;
  background: var(--text);
  color: #fff;
  font-size: 12px;
  padding: 4px 8px;
  border-radius: 4px;
}

@media (prefers-reduced-motion: reduce) {
  .bar, .x-label { transition: none; }
}
```

src/chart/chart.test.mjs

```
// Run: node src/chart/chart.test.mjs
import assert from 'node:assert/strict';
import { bandScale, linearScale, niceStep, niceTicks } from './scales.js';
import { binValues, parseNumbers } from './histogram.js';

assert.equal(niceStep(237, 5), 50);
assert.deepEqual(niceTicks([120, 237, 80]), { min: 0, max: 250, ticks: [0, 50, 100, 150, 200, 250] });
assert.deepEqual(niceTicks([-25, 60]).ticks, [-40, -20, 0, 20, 40, 60]);
assert.deepEqual(niceTicks([0.6]).ticks, [0, 0.2, 0.4, 0.6]); // no float noise
assert.deepEqual(niceTicks([]).ticks, [0, 0.2, 0.4, 0.6, 0.8, 1]); // empty / all zeros

assert.equal(linearScale([0, 250], [300, 0])(125), 150);
const x = bandScale(4, 400, 0.2);
assert.equal(x.step, 100);
assert.equal(x.bandwidth, 80);
assert.equal(x(1), 110);

assert.deepEqual(parseNumbers('1, 2 abc,3\n4.5,,'), { values: [1, 2, 3, 4.5], invalid: 1 });
```

```
const bins = binValues([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5);
assert.deepEqual(bins.map((b) => b.count), [2, 2, 2, 2, 3]); // max lands in the last bin
assert.equal(bins.at(-1).x1, 10);
assert.deepEqual(binValues([5, 5, 5], 3), [{ x0: 5, x1: 5, count: 3 }]);
assert.deepEqual(binValues([], 5), []);

console.log('chart tests passed');
```

12. Wrap-up

Edge cases handled

Empty data, all zeros, single value, negatives, only negatives, zero-height bars (hit area), decimals and float noise, any magnitude, narrow screens (label skipping), tooltip near edges, data change during hover, invalid histogram input, identical histogram values, max value binning, reduced motion.

Trade-offs to mention

- **SVG vs Canvas.** SVG gives DOM events, CSS, focus and accessibility for free, and is crisp at any zoom. It slows down past a few thousand nodes; Canvas scales to 100k+ bars but you must do hit-testing and accessibility yourself.
- **Measured pixels vs `viewBox`.** Measuring keeps text at 12px and re-lays out bars, at the cost of one extra render after mount. `viewBox` is zero-JS but scales text.
- **CSS transitions on `x/y/height`.** Very little code, and it works in current Chrome and Firefox where SVG geometry is a CSS property. For full control (stagger, enter/exit) you need a JS tween with `requestAnimationFrame`.
- **Tab stop per bar.** Simple and works with screen readers, but 30 bins means 30 tab stops. A roving tabindex (one tab stop, arrow keys between bars) is better for large charts.
- **Always including 0 in the domain.** Honest for bars (length encodes value). A line chart would not need it.
- **Bins by index id.** Fine because bins are recomputed from scratch; there is no "same bin" to animate between bin counts.

What to add with more time

- Roving tabindex with arrow keys and Home/End; a visually hidden `<table>` fallback of the data.
- Horizontal orientation and grouped / stacked series (band scale inside a band scale).
- Y-axis title, value labels on bars, legend, click-to-select callback (`onBarClick`).
- Automatic bin count (Sturges / Freedman-Diaconis) and "nice" bin edges (snap to `niceStep`).
- Canvas renderer for very large data; downsampling (bucket max per pixel).
- Component tests with React Testing Library (tooltip on focus, empty state) and visual regression snapshots.
- Dark mode via CSS variables (already used for colours).

13. Counter questions the interviewer may ask

Q1. Why not just use a chart library?

The task forbids it, but the real answer: a library is right for product dashboards. Writing it yourself shows you understand scales, SVG coordinates and accessibility, which you need anyway to customise or debug a library chart. The core here is ~150 lines.

Q2. Why SVG and not Canvas or divs?

Divs work for simple bars (flexbox + percentage heights) but axes, gridlines and negative values get awkward. SVG gives precise coordinates plus DOM events, CSS and focus. Canvas is for thousands of marks, and you lose all of that.

Q3. Why does the y range go from `innerH` to 0?

SVG's y-axis points down. Mapping the domain min to `innerH` (bottom) and max to `0` (top) flips it so larger values are drawn higher. Without the flip, bars grow from the top.

Q4. How do you draw a negative bar?

The rect starts at the higher of the two edges: `y = y(max(0, v))`, and `height = |y(v) - y(0)|`. For a positive value the top is `y(v)`; for a negative value the top is the zero line and the bar extends downward. SVG rects cannot have negative height, which is why we take the absolute value.

Q5. Walk me through `niceStep(237, 5)`.

Raw step 47.4. Power of ten below it is 10. Normalised fraction 4.74. Round up to the next of 1, 2, 5, 10, giving 5. Step = 50. Snap max up to 250. Ticks `0, 50, ..., 250`.

Q6. What if all values are 0, or the data is empty?

Domain would be `[0, 0]` and the scale divides by zero, giving `NaN` coordinates. `niceTicks` sets `max = 1` when `min === max`, so the axis is `0..1` and bars are flat. Empty data shows a placeholder instead of the SVG.

Q7. Why measure width with ResizeObserver instead of `window.resize`?

The chart's container can change size without the window resizing (sidebar collapse, grid layout, tabs). `ResizeObserver` watches the element itself, fires once on observe for the initial size, and is cheap. A window listener would miss those cases.

Q8. Why `useLayoutEffect` and not `useEffect`?

The observer is attached before the browser paints, so the first measured width arrives as early as possible and there is less chance of a visible empty frame. `useEffect` also works; the difference is one frame.

Q9. How do you stop x labels overlapping on a phone?

Each label needs about `minLabelWidth` px. If the slot (`x.step`) is narrower, show only every `ceil(minLabelWidth / step)`-th label. Alternatives: rotate labels 45 degrees, truncate with ellipsis, or switch to horizontal bars where labels have room.

Q10. Why store `activeId` rather than the hovered index?

After sorting or randomizing, index 6 is a different bar. The id stays attached to the same datum, so the tooltip and highlight follow it, and if the bar is removed the lookup returns nothing and the tooltip hides.

Q11. How did you make a zero-height bar hoverable?

A transparent rect the full height of the plot sits over each slot and takes the events and focus. The visible bar is only for drawing. It also makes thin bars easier to hit and gives a column highlight.

Q12. How is the chart accessible to screen reader and keyboard users?

The SVG has `role="group"` and an `aria-label` (the title). Each hit rect has `tabIndex={0}`, `role="img"` and an `aria-label` like `"Jul: 237"`. Focus shows the tooltip and a focus ring. The tooltip div is `aria-hidden` to avoid double reading. Next step: roving tabindex and a hidden data table.

Q13. What if the dataset has 1 million points?

One million SVG rects will freeze the page. A 1000px-wide chart can only show ~1000 bars anyway, so aggregate first: bucket points per pixel (min/max or mean), or bin them like the histogram, in a Web Worker if needed. Then render with Canvas if you still need thousands of marks. Also replace `Math.min(...values)` with a loop: spreading a huge array overflows the call stack.

Q14. What if the data comes from a slow API?

The chart stays a pure "data in, SVG out" component. The parent owns loading / error state and renders a skeleton of the same height (so the layout does not jump) or an error with retry. When new data arrives, keyed bars animate to the new values.

Q15. How do the animations work, and what would you change?

Bars keep the same DOM node thanks to `key={d.id}`, and CSS transitions animate `x`, `y` and `height` when attributes change. It is tiny but you cannot do enter/exit or staggering. For that, interpolate values in JS with `requestAnimationFrame`, or use the Web Animations API. `prefers-reduced-motion` turns transitions off.

Q16. Why does the domain always include zero?

Bar length encodes value. If the axis started at 80, a bar of 120 would look three times a bar of 93. Including zero keeps the visual ratio honest. Line charts can zoom to the data range because they encode position, not length.

Q17. How would you build horizontal bars?

Swap the roles: the band scale runs along y (rows), the linear scale along x with a normal (unflipped) range `[0, innerW]`. Bar `x = x(min(0, v))`, `width = |x(v) - x(0)|`. Labels go on the left, which also fixes long-label overlap.

Q18. How would you add grouped bars (two series per month)?

Nested band scales: the outer band scale positions each month, an inner band scale with `count = seriesCount` and `width = outer.bandwidth` positions each series inside it. The y scale's domain covers all series. Add a legend with series colours.

Q19. Histogram: why clamp the bin index?

`floor((max - min) / width)` equals `binCount` for the maximum value, which is one past the last bin. Clamping puts the max into the last bin, matching the convention that the last bin is closed on both sides `[x0, x1]`.

Q20. How would you choose the number of bins automatically?

Sturges' rule: `ceil(log2(n) + 1)`, good for normal-ish data. Freedman-Diaconis: bin width `2 * IQR / cbrt(n)`, robust to outliers. Then snap bin edges to `niceStep` so labels read `100-125` instead of `100-125.4`.

Q21. Why are the scale functions outside React?

They are pure math: same input, same output, no DOM. That makes them unit-testable with `node:assert` in milliseconds and reusable for any chart type. The component just wires them to SVG.

Q22. Do you need `useMemo` for the scales?

Not for a few hundred bars; the math is microseconds and recomputing keeps code simple. Memoise the expensive parts instead: sorting and binning are wrapped in `useMemo` keyed on their inputs. Profile before memoising more.